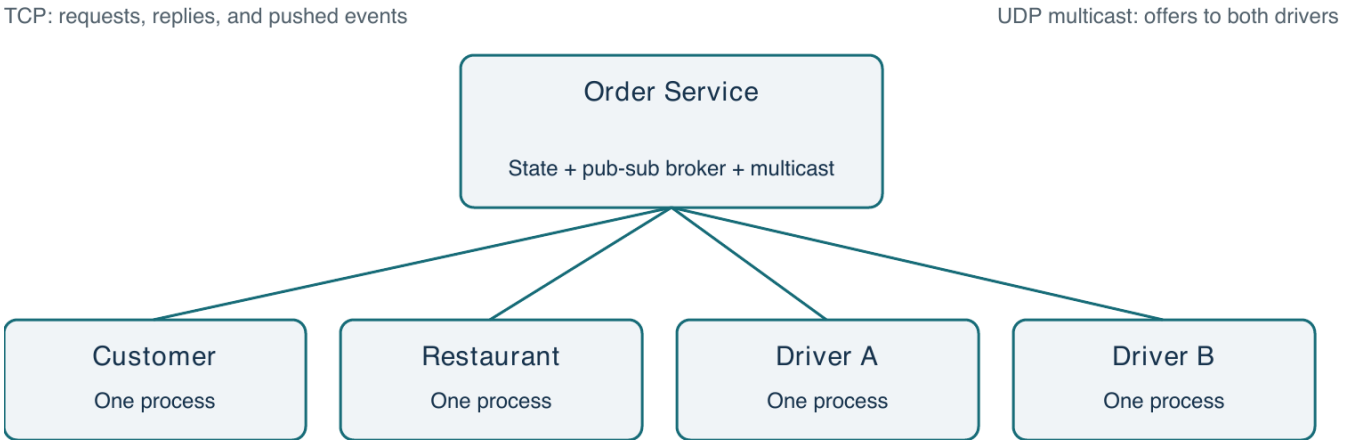
Milestone 2 Communication Report

Didi Food Delivery

1. Communication model:

- This milestone implements communication among a customer, a restaurant, two driver nodes, and a central order service. The demonstration covers request-reply interactions, JSON marshalling and unmarshalling, publish-subscribe notifications, multicast group communication, and timestamped logs. The interface is terminal-based, so message exchange remains visible.

- Four source files run as five independent processes on one macOS computer. Loopback sockets connect the processes; this is a local demonstration of distributed communication patterns, not a deployment across multiple physical computers. Cooking and delivery are simulated by pressing Enter.



- Clients open a TCP connection to 127.0.0.1:6000, send one JSON command, read one reply, and close the connection. PLACE_ORDER, ACCEPT_ORDER, UPDATE_ORDER_STATUS, and COMPLETE_DELIVERY use this interaction. Replies indicate confirmation, successful assignment, rejection, success, or an error.

- The customer sends an order request, the restaurant reports food readiness, and the assigned driver reports delivery. The service processes these commands and sends replies. A separate persistent TCP connection delivers order events to subscribers, while UDP multicast distributes an order offer to both drivers.

2. Message formats:

- Commands use an action field. Replies use status and message, with order_id included where the handler returns it. Events use event_type, order_id, payload, and published_at. The examples below follow the current source; timestamps are illustrative values, not captured test output.

2.1 Customer request and service response


```
{
 "action": "PLACE_ORDER",
 "order_id": 8801,
 "customer_name": "Jordan Smith",
 "restaurant": "DiDi Express Burger",
 "items": [
  "Bacon Cheese Burger",
  "Vanilla Milkshake, American Fries"
 ],
 "timestamp": "2026-10-12T00:00:00.000"
}
```



```
{
 "status": "ORDER_CONFIRMED",
 "order_id": 8801,
 "message": "Order #8801 accepted",
 "timestamp": "2026-10-12T00:00:00.001"
}
```


2.2 Pub-sub event


```
{
 "event_type": "ORDER_READY_FOR_PICKUP",
 "order_id": 8801,
 "payload": {
  "status": "READY_FOR_PICKUP",
  "restaurant": "DiDi Express Burger",
  "assigned_driver": "An"
 },
 "published_at": "2026-10-12T00:00:28.395"
}
```


2.3 Multicast offer


```
{
 "communication_type": "GROUP_MULTICAST",
 "target_group": "DRIVERS_ZONE_A",
 "order_id": 8801,
 "restaurant": "DiDi Express Burger",
 "destination": "1250 Bellflower Blvd, Long Beach, CA",
 "timestamp": "2026-10-12T00:00:00.002"
}
```


- For TCP, the serialized JSON has a final newline. A displayed b'...' is Python's representation of bytes, not an extra character sent as part of the JSON document. order_id links the demo requests and events; separate request IDs are not implemented.

3. Marshalling method:

Python dictionary

-> json.dumps(data)

-> append "\n"

-> encode("utf-8")

-> send bytes through a socket

Received bytes

-> read one complete newline-terminated message

-> decode("utf-8")

-> json.loads(...)

-> Python dictionary

- The function receive_message() uses a persistent buffered reader for subscription traffic. This allows fragmented TCP data and multiple consecutive messages to be decoded separately. Messages exceeding 16,389 bytes or lacking the required ending newline are rejected. UDP retains each datagram boundary.

- Before marshalling, the message is a Python dictionary. json.dumps() converts it to JSON text, and encode("utf-8") converts that text to bytes for the socket. On the receiving side, decode("utf-8") restores the JSON text and json.loads() restores the dictionary. SHOW_DETAILS logs make these transformations visible.

4. Explanation of sockets RPC or RMI choice:

- Sockets were selected because they expose message serialization, connection handling, timeouts, and message boundaries directly, while requiring no external Python packages. TCP provides an ordered byte stream for commands and responses. The application adds newline framing because TCP does not preserve application-message boundaries.

- This project implements socket communication directly through Python's standard library. It does not implement RPC or RMI. An RPC interface would let a client invoke a remote procedure through a call abstraction, while RMI applies that idea to methods on remote objects. Direct sockets fit this demonstration because the code explicitly shows serialization, request and reply exchange, persistent event connections, and multicast group membership.

5. Pub-sub design:

- Each subscribing process opens a separate, persistent TCP connection and sends SUBSCRIBE_UPDATES. The service returns a SUBSCRIBED acknowledgment before registering the socket for later event delivery. This confirms readiness without relying on a fixed startup sleep.

- The restaurant publishes a readiness update indirectly: it sends a status command to the service, which validates the update and publishes ORDER_READY_FOR_PICKUP. The service similarly publishes ORDER_CREATED, ORDER_TAKEN, and ORDER_DELIVERED. Customer, restaurant, and driver processes are subscribers; the driver that loses the assignment exits the single-order demo.

- There is one shared event channel. The service sends events to connected subscribers, and each application filters or handles events locally using order_id and event_type. There is no topic-routing service, retained-event store, or replay for late subscribers.

6. Group-communication design:

- The service sends one order-offer datagram to multicast group 224.1.1.1, port 5007. Both driver sockets bind to the receiving port and join the group with IP_ADD_MEMBERSHIP. The multicast interface is explicitly set to 127.0.0.1 so this demonstration remains on the local computer.

- The multicast address identifies the receiver group, while the interface selects the local network path. Unlike the persistent TCP pub-sub connections, group delivery uses UDP datagrams. Both drivers must start before the offer is sent. Delivery is not acknowledged, retried, or guaranteed.

- Drivers claim an offered order through TCP request-reply rather than through multicast. The service checks the assignment under a local thread lock, assigns one driver, and publishes ORDER_TAKEN. The other driver receives that event and exits this single-order demonstration.

7. Screenshots or logs showing messages:

The following excerpts are clipped from the screenshots supplied during the actual terminal demonstration. Their text has not been rewritten. The panels document kitchen order receipt, both multicast receivers, and serialized request-reply data.

Figure 1: Restaurant connects to Order Service server and joins the PUB/SUB for order status.



Figure 2: Two drivers join after the restaurant and also join the PUB/SUB to receive order status information.

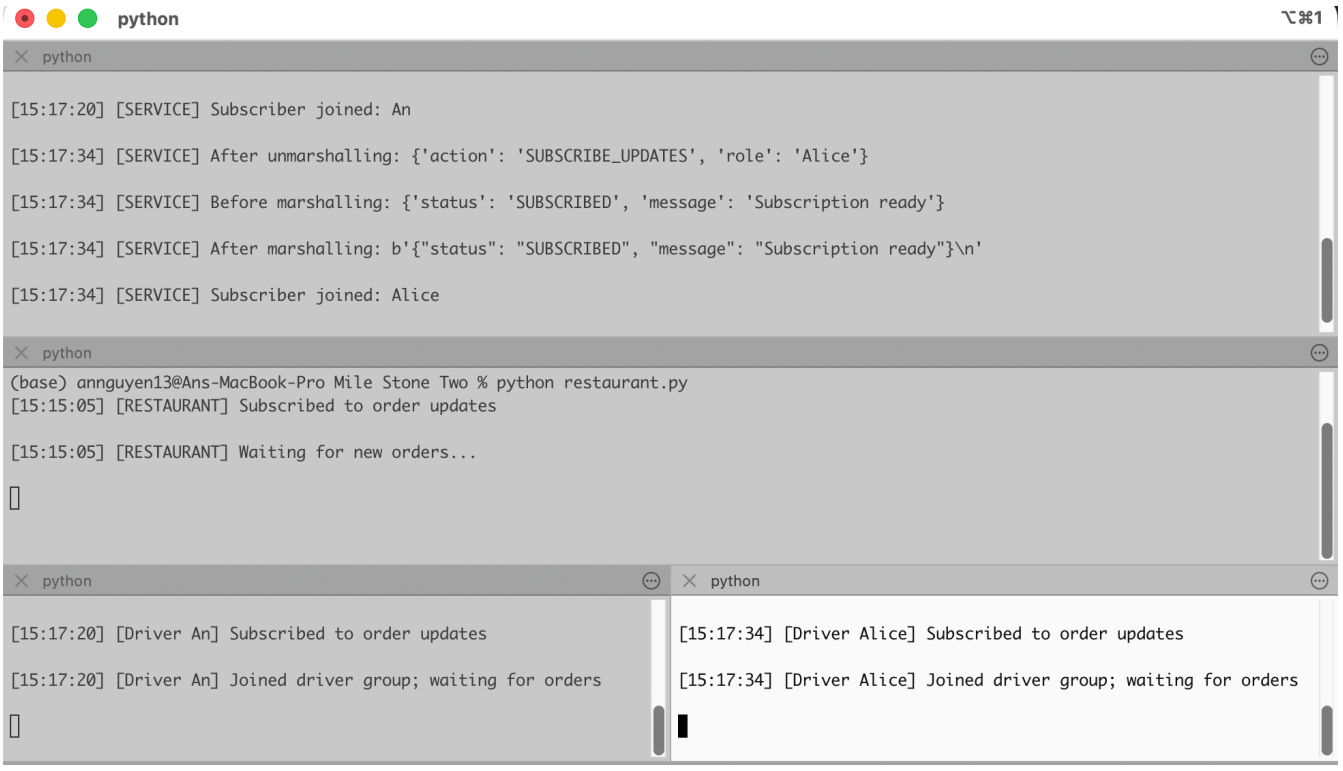


Figure 3: Customer connects to the order_service server and sends an order to the server, then joins the PUB/SUB channel to receive the status of the order. Once the order service gets the order, it will MULTICAST to the two drivers as well as send the order information to the restaurant so that they can prepare.

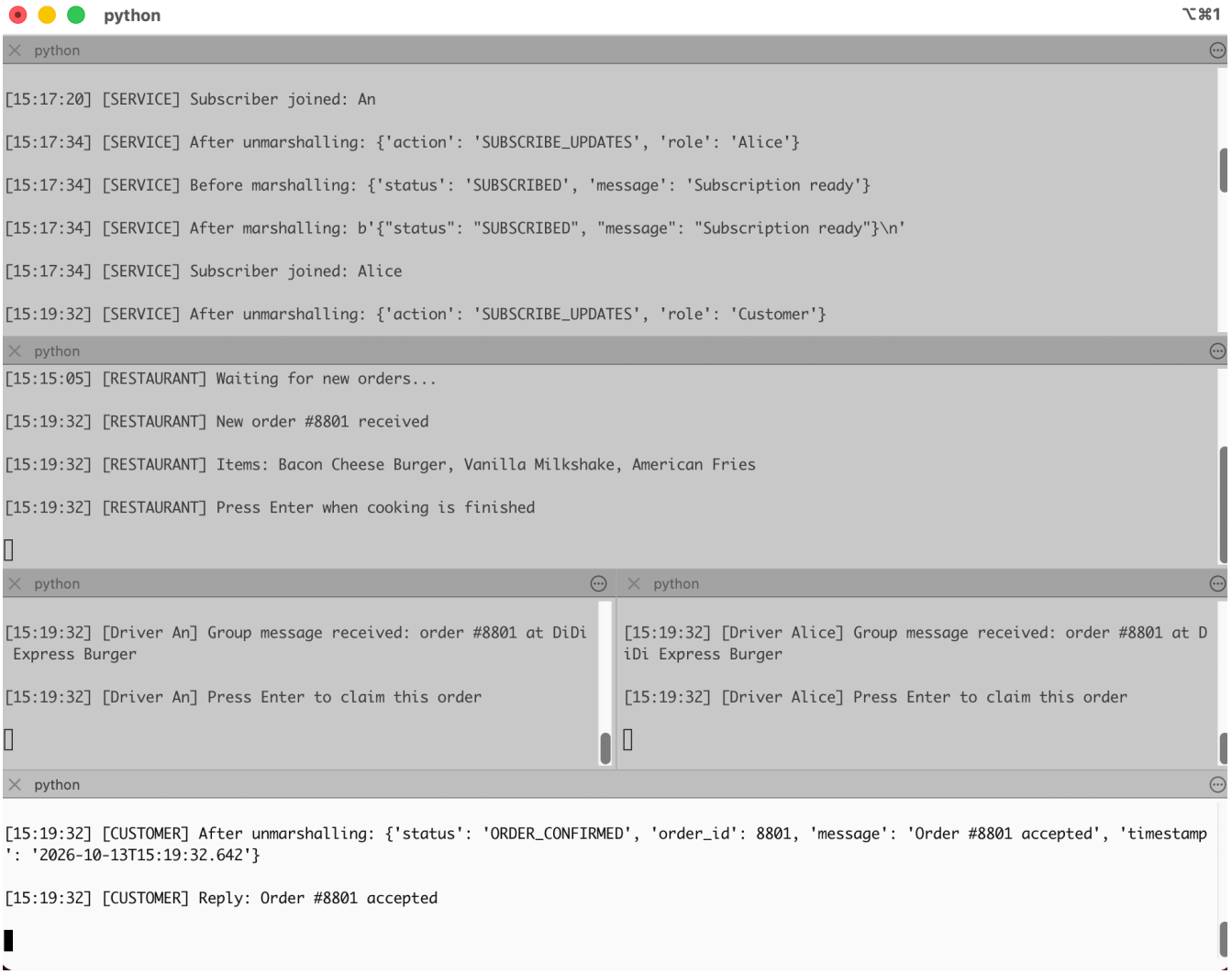
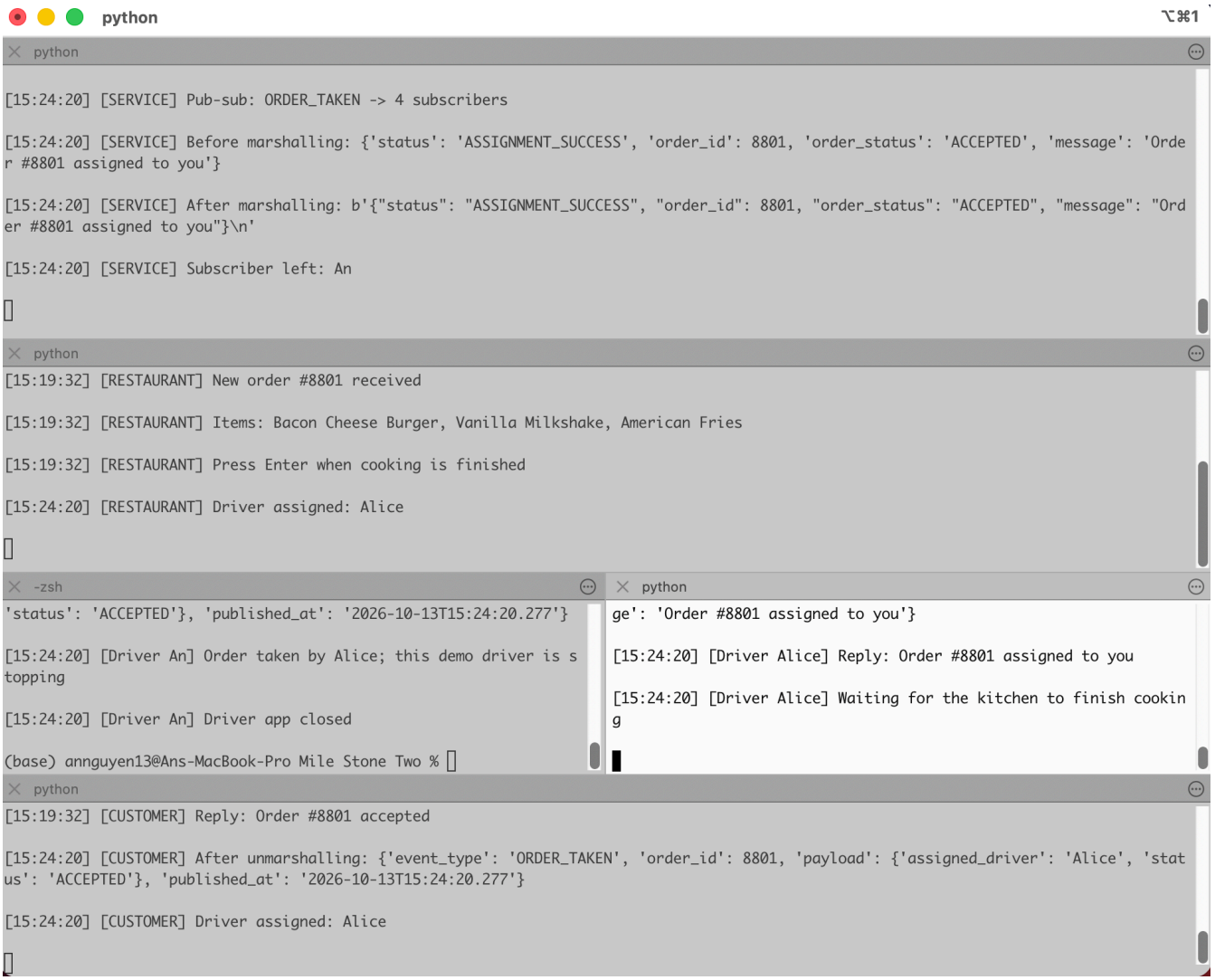


Figure 4: If driver Alice claims the delivery for that order by clicking ENTER, driver An's process will terminate and leave the PUB/SUB channel, and driver Alice will wait for the order to be cooked and delivered after the restaurant hits ENTER, telling the order has been done and is ready for pickup.



Figures 5: Once the restaurant hits ENTER to signal driver Alice that the order is ready for pickup, driver Alice will pick up and hit ENTER when the order is successfully delivered to the customer.

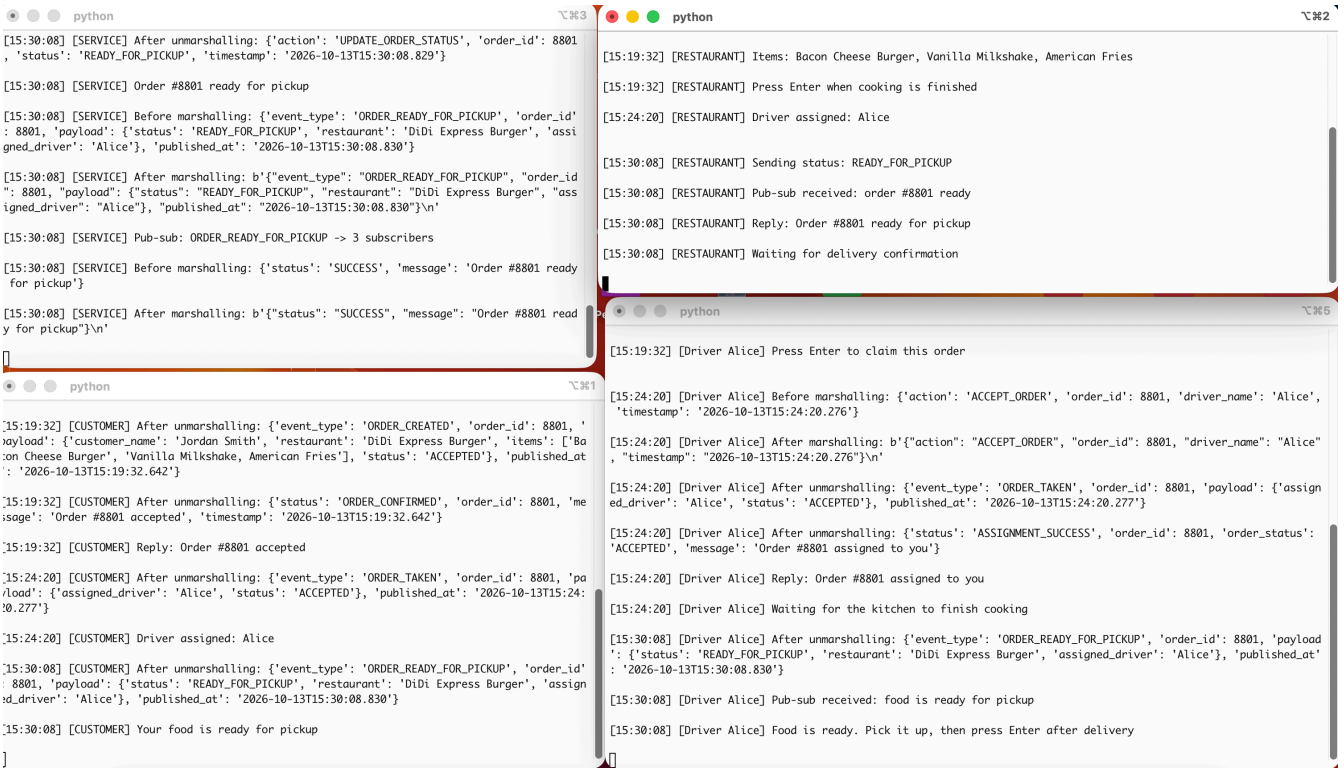


Figure 6: Once driver Alice hits ENTER, telling the order service server that the order has been delivered successfully to the customer. The order service server's terminal will show that all PUB/SUB members, including the Restaurant, Driver Alice, and Customer, have left its PUB/SUB channel, which was used for receiving order status between processes. On the other hand, the restaurant's terminal will show that order #8801 was delivered, and the Restaurant app closes, while the Customer's terminal will show "Your food has arrived, Order completed, and Customer app closes," and Driver Alice's terminal will also show that "order #8801 delivered and the Driver app closed".

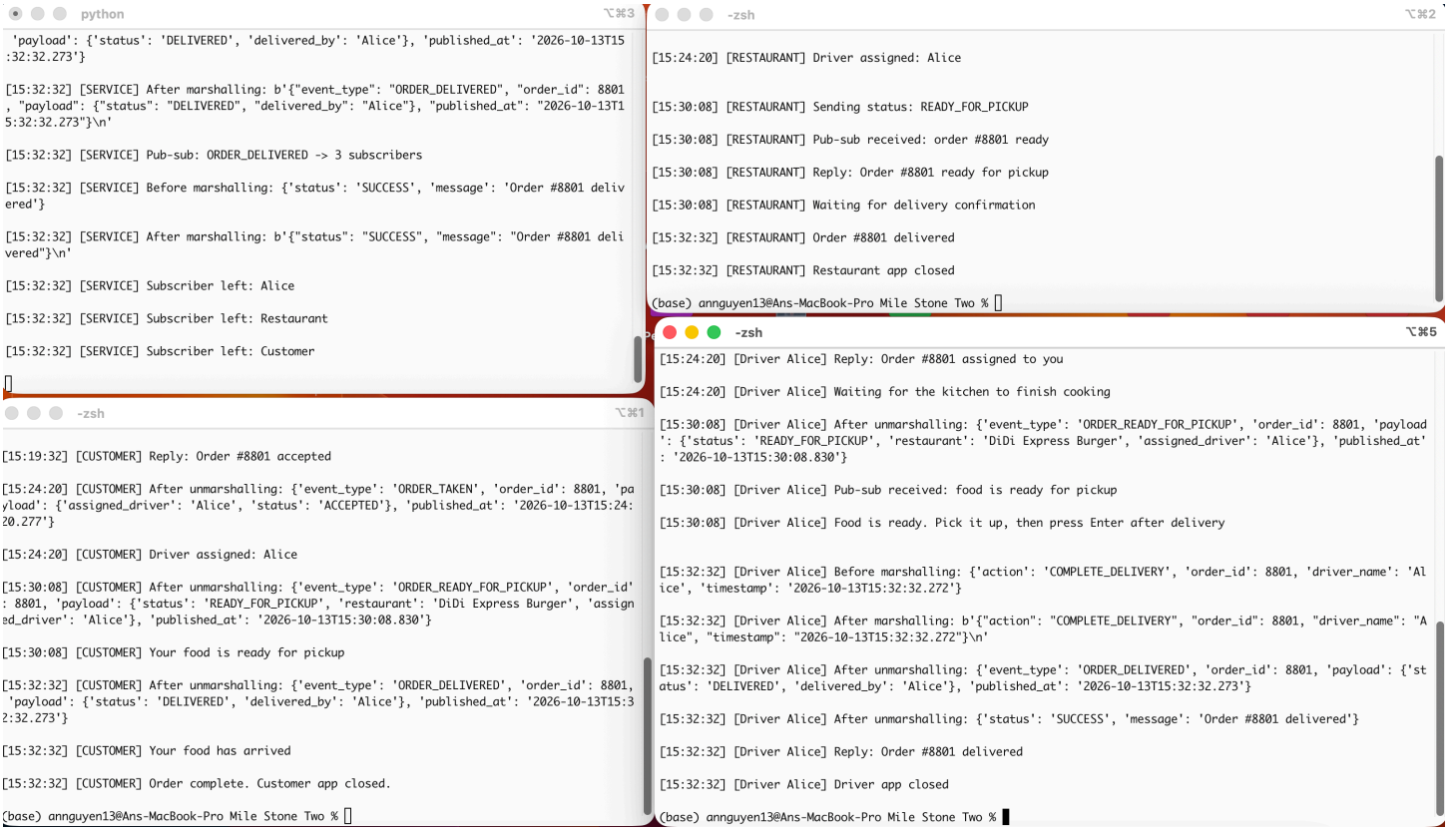


Figure 7: Demonstrates service-disconnection detection and graceful client termination. When the order service stops, the remaining clients detect the closed TCP connection and exit without reporting successful delivery. Automatic recovery and failover are not implemented.

